Introducing Call: Because Calling a Function Shouldn't Become Your Next Project

FOR IMMEDIATE RELEASE
The function was right there. We've made arrangements.

A modern take on Epicor dynamic dispatch. Based on FunctionRunner.

October 8, 2026; updated October 10, 2026 - I built **Call** because I had a library name, a function name, and some arguments, and I foolishly believed that ought to be enough to call a function.

Apparently, I had brought common sense to a framework discussion.

Call lets you select and invoke Epicor Functions at runtime. Tell it which library, which function, what inputs, and how you want errors handled. It runs the function and gives you the outputs.

That's the pitch. I could call it an "adaptive enterprise execution orchestration platform," but then I'd have to hate myself.

```
var (success, outputs, details) = EfxLib.CallLib.Call(
    "MyLibrary", "MyFunction", new object[] { myDataSet, 42 }, "Throw");
```

There. The launch webinar has concluded.

Call is the product. `CallLib` is the Function library. `CallUBAQ` is the dispatch BAQ. We have now exhausted the necessary branding discussion.

Server callers pass real objects. DataSets. Epicor tablesets. The actual things you already have. They stay objects, without being serialized into JSON, sent on a character-building journey, and reconstructed into something that hopefully resembles what you started with.

REST callers get positional or named JSON arguments, bound against the target Function's declared signature. Named arguments are particularly useful if you've decided that remembering what the seventh parameter means is a poor substitute for having a life.

Making several calls in one request? `CallLib.Get` lets you acquire reusable calling methods through the supplied helper. Prefer a scalar or tuple? `Packed` handles that.

You have options. None require a steering committee.

Error handling comes in two explicit flavors:

- **Throw:** Raise an Epicor business exception. Something failed. Act accordingly.

- **Return:** Return a failure result with details so your application can decide what to do. You are the developer. This seems like a reasonable allocation of responsibility.

Successful outputs are the same in either mode. Failures include structured details about the target, stage, status, elapsed time, and reported error. Useful information, for those nostalgic for that sort of thing.

Copyright (c) 2026 Kevin P. Lincecum | MIT License

Call does **not** automatically retry an operation just because the first attempt ended awkwardly. "Perhaps it already posted; let's do it again" is how accounting gets involved.

If Call cannot establish the outcome, it reports **Unknown**. It does not convert uncertainty into a green checkmark to protect the dashboard's feelings.

REST consumers using Return mode must still check `oSuccess`. HTTP 200 means you received a response. It does not mean the business operation succeeded, your data is correct, or your implementation deserves a commemorative plaque.

Installation uses a matching **CallLib + CallUBAQ** package pair for your Epicor/ICE package version. Packages are provided for **4.3.200**, **5.1.100**, and **5.2.100**, along with the SDK guide, a consumer helper, installation guidance, and an AI integration kit.

Pick the version that matches your environment. "I downloaded the bigger number" is not a compatibility assessment, despite its impressive adoption rate.

The packages ship with a dummy company and me as owner. Import them, then set your company, ownership, and security properly. A press release does not configure your server. Neither does putting "enterprise-ready" in bold and hoping nobody asks follow-up questions.

Call does one useful thing: it lets your code choose an Epicor Function at runtime and call it through a consistent interface.

It will not digitally transform your organization. It will not align your stakeholders. It has no strategic vision for your fiscal quarter.

It will call the bloody function.

Call. You already know what you want to run. Get on with it.

Copyright (c) 2026 Kevin P. Lincecum. Licensed under the MIT License.

MIT License

Copyright (c) 2026 Kevin P. Lincecum

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.